# دليل التثبيت — خزانة-DZ

> *تطبيق ذكي لإدارة المخزون — متعدد القطاعات*

---

## فهرس المحتويات


1. المتطلبات الأساسية
2. التثبيت — وضع التطوير
3. التثبيت — وضع الإنتاج (Docker)
4. التثبيت — تطبيق سطح المكتب المستقل
5. التحقق من التثبيت
6. استكشاف الأخطاء وإصلاحها


---

## 1. المتطلبات الأساسية

### البرامج المطلوبة

| البرنامج | الحد الأدنى للإصدار | التحميل |
|---|---|---|
| **Node.js** | x (LTS).18 | https://nodejs.org/ |
| **npm** | x.9 (مضمّن) | مضمّن مع Node.js |
| **Docker** | x.24 | https://www.docker.com/products/docker-desktop |
| **Git** | x.2 | https://git-scm.com/ |

### التحقق من المتطلبات

افتح محطة PowerShell ونفّذ:

```
node --version      # يجب أن يعرض v18.x أو أعلى
npm --version       # يجب أن يعرض x.9 أو أعلى
docker --version    # يجب أن يعرض x.24 أو أعلى
git --version       # يجب أن يعرض x.2 أو أعلى
```

### المنافذ المستخدمة

| الخدمة | المنفذ | ملاحظة |
|---|---|---|
| PostgreSQL | 5433 | حاوية Docker (معيّنة من 5432) |
| Redis | 6379 | حاوية Docker |
| واجهة API | 3002 | NestJS |
| الواجهة الأمامية | 5173 | خادم Vite (وضع التطوير) |

> ⚠️ *تأكد أن هذه المنافذ غير مستخدمة على جهازك.*

---

## 2. التثبيت — وضع التطوير

### الخطوة 1: استنساخ المشروع

```
Khazane-DZ <رابط-المستودع> git clone
cd Khazane-DZ
```

### الخطوة 2: تشغيل حاويات Docker (PostgreSQL + Redis)

```
npm run docker:up
```

يقوم هذا الأمر بتشغيل:

- **PostgreSQL 16** على المنفذ `5433` (مستخدم: `khazane` ، كلمة المرور: `khazane_secret` ، قاعدة البيانات: `khazane_db` )
- **Redis 7** على المنفذ `6379`

للتحقق من تشغيل الحاويات:

```
docker ps
```

يجب أن ترى `khazane-postgres` و `khazane-redis` بحالة `Up` .

### الخطوة 3: تثبيت تبعيات الخادم الخلفي

```
cd backend
npm install
```

### الخطوة 4: ضبط متغيرات البيئة

ملف `backend/.env` مُعدّ مسبقاً:

```
# قاعدة البيانات
DATABASE_URL=postgresql://khazane:khazane_secret@localhost:5433/khazane_db

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379

# JWT
JWT_SECRET=change-me-in-production
JWT_EXPIRES_IN=1d

# التطبيق
PORT=3002
NODE_ENV=development
```

> ⚠️ ***في بيئة الإنتاج***، *يجب تغيير* `JWT_SECRET` *إلى قيمة عشوائية آمنة وتغيير كلمات مرور قاعدة البيانات.*

### الخطوة 5: تنفيذ ترحيلات قاعدة البيانات

```
npx prisma migrate dev
```

ينشئ هذا الأمر جميع الجداول في PostgreSQL ويولّد عميل Prisma.

### الخطوة 6: تغذية قاعدة البيانات (seed)

```
npx prisma db seed
```

البيانات التي ينشئها الـ seed:

- **مستخدم مسؤول**: `admin@khazane.dz` / كلمة المرور `admin123`
- **موقع**: «المستودع الرئيسي» (رمز `ENT-01`)، نوع Warehouse، عنوان «المنطقة الصناعية، الجزائر»
- **منطقة**: «الممر 1» (رمز `A1`)، نوع AISLE
- **5 مواقع تخزين**: `A1-01` إلى `A1-05` (رف 1 إلى 5)، سعة قصوى 100
- **فئة**: «عام»
- **تعيين**: المسؤول معيّن للموقع الرئيسي

### الخطوة 7: تشغيل الخادم الخلفي

```
npm run start:dev
```

يبدأ خادم API على `http://localhost:3002` .
وثائق Swagger متاحة على `http://localhost:3002/api` .

### الخطوة 8: تثبيت تبعيات الواجهة الأمامية

افتح **محطة جديدة**:

```
cd frontend
npm install
```

### الخطوة 9: تشغيل الواجهة الأمامية

```
npm run dev
```

تبدأ الواجهة الأمامية على `http://localhost:5173` .

### ملخص — الأوامر السريعة (من المجلد الجذر)

```
# 1  .npm run docker:up          تشغيل PostgreSQL + Redis
# 2  .cd backend && npm install  تبعيات الخادم الخلفي
# 3  .npx prisma migrate dev     الترحيلات
# 4  .npx prisma db seed         البيانات الأولية
# 5  .npm run start:dev          تشغيل الخادم الخلفي
# في محطة جديدة:
```

```
# 6 cd frontend && npm install .تبعيات الواجهة الأمامية
# 7 npm run dev .تشغيل الواجهة الأمامية
```

---

## 3. التثبيت — وضع الإنتاج (Docker)

### بناء الواجهة الأمامية

```
cd frontend
npm ci
npm run build
```

يتم إنشاء البناء في `/frontend/dist` .

### بناء الخادم الخلفي

```
cd backend
npm ci
npx nest build
```

يتم إنشاء البناء في `/backend/dist` .

### تشغيل الخادم الخلفي في وضع الإنتاج

```
cd backend
"env:NODE_ENV="production$
"env:DATABASE_URL="postgresql://khazane:khazane_secret@localhost:5433/khazane_db$
"السر-الآمن-للإنتاج"=env:JWT_SECRET$
"env:PORT="3002$
"env:SERVE_STATIC="true$
node dist/main.js
```

مع `SERVE_STATIC=true` ، يقدم الخادم الخلفي الواجهة الأمامية المبنية تلقائياً. التطبيق الكامل متاح على `http://localhost:3002` .

---

## 4. التثبيت — تطبيق سطح المكتب المستقل

تطبيق سطح المكتب هو ملف تنفيذي مستقل لنظام Windows يتضمن PostgreSQL والخادم الخلفي والواجهة الأمامية.

### المتطلبات الإضافية

- اتصال بالإنترنت (لتحميل PostgreSQL المحمول ~200 ميغابايت)
- حوالي 1 غيغابايت من مساحة القرص الحرة

### تشغيل البناء الآلي

من المجلد الجذر للمشروع:

```
scripts\build-desktop.ps1\.
```

ينفذ هذا البرنامج النصي 6 خطوات تلقائياً:

| الخطوة | الإجراء |
|---|---|
| 1/6 | تحميل PostgreSQL 16 المحمول |
| 2/6 | بناء واجهة React الأمامية |
| 3/6 | بناء خادم NestJS الخلفي |
| 4/6 | تجميع برنامج الـ seed |
| 5/6 | تثبيت تبعيات Electron |
| 6/6 | التغليف بـ electron-builder (NSIS) |

### خيارات البرنامج النصي

```
# تخطي تحميل PostgreSQL (إذا كان موجوداً)
scripts\build-desktop.ps1 -SkipPgDownload\.

# إنشاء مجلد غير مغلّف (للاختبار)
scripts\build-desktop.ps1 -DirOnly\.
```

### النتيجة

يتم إنشاء مثبّت ( `.exe` ) Windows في مجلد `/dist-desktop` .

### التثبيت على جهاز العميل

1. تشغيل مثبّت `exe.`
2. اتباع معالج التثبيت
3. يبدأ التطبيق بشاشة تحميل أثناء تهيئة قاعدة البيانات المدمجة
4. لا يلزم تثبيت PostgreSQL أو Node.js على جهاز العميل

---

## 5. التحقق من التثبيت

### اختبار تسجيل الدخول

1. افتح المتصفح على `http://localhost:5173` (تطوير) أو `http://localhost:3002` (إنتاج)
2. سجّل الدخول بـ:
   - **البريد الإلكتروني**: `admin@khazane.dz`
   - **كلمة المرور**: `admin123`
3. يجب أن تصل إلى لوحة التحكم

### اختبار واجهة API

```
# الحصول على رمز JWT
$body = '{"email":"admin@khazane.dz","password":"admin123"}'
$response = Invoke-RestMethod -Uri http://localhost:3002/api/v1/auth/login -Method POST -
Body $body -ContentType "application/json"
$token = $response.access_token
```

```
# اختبار واجهة المنتجات
Invoke-RestMethod -Uri http://localhost:3002/api/v1/products -Headers @{ Authorization =
"Bearer $token" }
```

### أدوات مفيدة

```
# فتح Prisma Studio (واجهة مرئية لقاعدة البيانات)
npm run db:studio

# عرض سجلات حاويات Docker
docker logs khazane-postgres
docker logs khazane-redis
```

---

## 6. استكشاف الأخطاء وإصلاحها

### المنفذ 5433 مستخدم بالفعل

```
# البحث عن العملية التي تستخدم المنفذ
netstat -ano | findstr :5433
# إيقاف الحاوية الموجودة
docker stop khazane-postgres
docker rm khazane-postgres
# إعادة التشغيل
npm run docker:up
```

### خطأ "Prisma "Can't reach database server

تأكد من تشغيل Docker وأن حاوية PostgreSQL تعمل:

```
docker ps | findstr khazane-postgres
```

إذا لم تظهر الحاوية في القائمة:

```
npm run docker:up
```

### الخادم الخلفي لا يبدأ (المنفذ 3002 مشغول)

```
netstat -ano | findstr :3002
F/ <رقم_العملية> taskkill /PID
```

### إعادة تعيين قاعدة البيانات

```
cd backend
npx prisma migrate reset
```

```
npx prisma db seed
```

> ⚠️ *هذه العملية تحذف جميع البيانات الموجودة.*

### إعادة إنشاء عميل Prisma

إذا ظهرت أخطاء متعلقة بـ Prisma بعد تغيير المخطط:

```
cd backend
npx prisma generate
```

---

*تم إنشاء هذا المستند لـ Khazane-DZ الإصدار 0.1.0*